Пирамидальная сортировка HeapSort

Пирамида Хеопса

Что такое "пирамида" или "куча"?

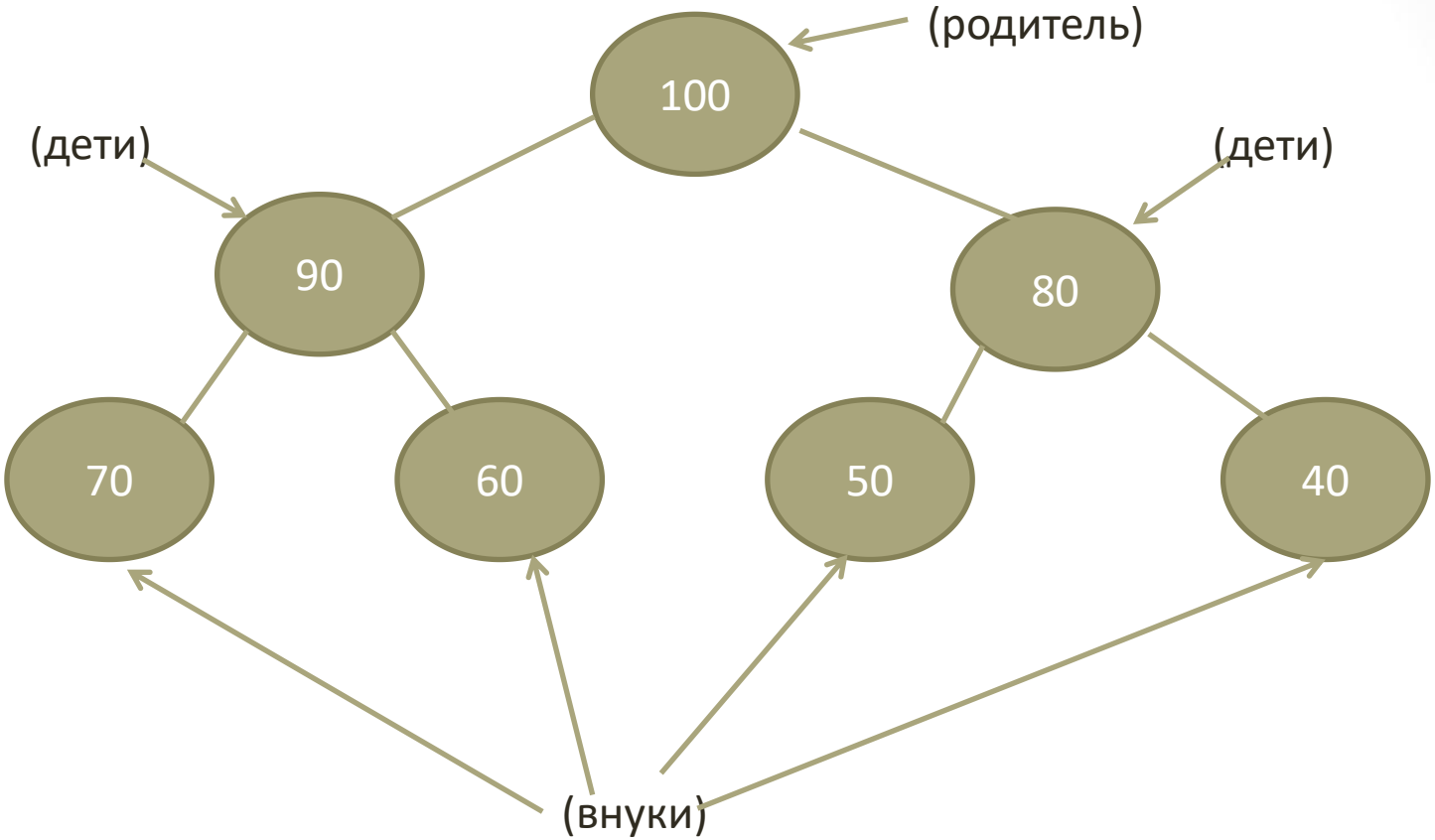
Пирамида (binary heap) — это структура данных, представляющая собой объект-массив, который можно рассматривать как почти полное бинарное дерево

У него два правила:

Правило формы: Дерево всегда аккуратное — все ряды заполнены подряд, слева направо, без дырок. Последний ряд может быть неполным, но всё равно заполняется слева.

Правило старшинства: Каждый родитель (верхний элемент) всегда больше или равен своим детям (тем, кто ниже). Поэтому на самом верху всегда лежит самый большой элемент.

Пример



Как хранить такую пирамиду в памяти компьютера?

Всю заключатся в том, что нам не нужно рисовать деревья. Всё хранится в обычном списке (массиве), и работает это по волшебным формулам.

Если у нас есть элемент с номером i (начиная с нуля):

Его левый ребёнок лежит по адресу: 2 * i + 1

Его правый ребёнок лежит по адресу: 2 * i + 2

Его родитель лежит по адресу: (i - 1) / 2 (целая часть)

Пример: Берём элемент №0 (это 100). Его дети:

Левый: 2*0+1 = 1 (это 90)

Правый: 2*0+2 = 2 (это 80)

Поддержка свойства пирамиды

Самая главная процедура называется "просеивание вниз". Представь, что в пирамиду попал слишком мелкий камушек, который оказался наверху. Правила нарушены! Нужно его просеять вниз, чтобы он опустился на своё место.

Как это работает (по шагам):

1.Смотрим на родителя и двух его детей.

2.Ищем, кто из этих троих самый большой.

3.Если родитель и так самый большой — отлично, ничего не делаем, просеивание закончено.

4.Если самый большой — один из детей, меняем их с родителем местами.

5.Теперь наш родитель опустился на один уровень вниз. Снова смотрим на его новых детей и повторяем всё с шага 1, пока он не окажется там, где он будет не меньше своих детей.

Как построить пирамиду из обычного списка?

У нас есть просто набор чисел в беспорядке. Как сделать из него правильную пирамиду?

Очень просто: Берём и просеиваем все элементы, у которых есть дети, начиная с самого нижнего такого родителя и поднимаясь вверх к корню.

Визуализация:
Был список [8, 3, 9, 6, 1, 2, 5]. Мы поколдовали (просеяли где надо) и получили [9, 6, 8, 3, 1, 2, 5]. Теперь наверху самый большой — 9. Пирамида готова!

Алгоритм

Вся сортировка состоит из двух шагов:

Шаг 1. Строим пирамиду из нашего беспорядочного списка (как на прошлом слайде).

Шаг 2. Сортируем:

Берём самый большой элемент (он всегда наверху, под индексом 0) и меняем его местами с последним элементом в списке. Теперь этот последний элемент стоит на своём месте — он самый большой и больше нас не интересует.

Оставшаяся часть списка (без последнего элемента) — это уже не совсем пирамида, потому что наверху теперь какой-то левый элемент. Применяем к нему просеивание, чтобы опять получить правильную пирамиду.

Повторяем шаги 1 и 2, пока вся пирамида не рассосётся, а список не станет отсортированным.

Анализ сложности

По времени:

Просеивание одного элемента занимает примерно $\log_2 N$ шагов. Для 1000 элементов это около 10 шагов.

Всего мы делаем N таких просеиваний.

Итог: Примерно $N \cdot \log N$ операций. Для 1000 элементов это около 10 000 операций. Для сравнения, простые сортировки (пузырьком) делали бы миллион операций.

По памяти:

Всё делается прямо в том же списке, никаких дополнительных больших массивов не нужно.

Плюсы и минусы

Плюсы:

Стабильная скорость: Всегда работает за N·log N, даже если кто-то специально подберёт для неё плохие числа.

Не занимает много памяти: Всё делает на месте, без копий.

Надёжность: Можно использовать, когда время выполнения критично и сюрпризы не нужны.

Минусы:

Неустойчивость: Если у нас есть два одинаковых числа, их порядок может поменяться. Обычно это не страшно, но в некоторых задачах важно.

На практике медленнее: Хоть математически всё красиво, в реальной жизни быстрая сортировка (quick sort) часто работает.

Сравнение

Алгоритм	Лучший случай	Средний случай	Худший случай	Доп. память
Пирамидальная	N·log N	N·log N	N·log N	O(1)
Быстрая	N·log N	N·log N	N^2 (страшно медленно!)	log N
Пузырьковая	N	N^2	N^2	O(1)
Вставками	N	N^2	N^2	O(1)

Пузырьковая и вставками — простые, но для больших массивов работают очень медленно (квадратичная сложность).

Быстрая — в среднем самая быстрая, но если выбрать плохие числа, она может дико тормозить (это её "плохой случай").

Пирамидальная — единственная среди быстрых, которая и память не ест, и всегда работает одинаково, без сюрпризов.

Итоги

Пирамидальная сортировка использует хитрую структуру "куча", где самый большой элемент всегда наверху.

Алгоритм — это просто инструкция. Наш алгоритм: "построй пирамиду, потом меняй верхушку с последним и просеивай".

Работает всегда за $N \cdot \log N$, без сюрпризов.

Не требует дополнительной памяти.

Немного медленнее быстрой сортировки в жизни, но гораздо надёжнее.